

Certificate

Name: Shahid Sakeem

Class: 4th Sem

Roll No: 240252

Exam No:

Institution GDC Dooaru

This is certified to be the bonafide work of the student in the
Computer Application Laboratory during the academic
year 2025/2026

No. of practicals certified 18 out of 21 in the
subject of DBMS

.....
Teacher In-charge

.....
Examiner's Signature

.....
Principal

Date:

Institution Rubber Stamp

(N.B: The candidate is expected to retain his/her journal till he/she passes in the subject.)

Aim: To study and implement DDL commands in SQL for creating tables.

Theory: DDL (Data Definition Language) commands are used to define and manage the structure of database objects such as tables.

The main DDL commands are:

- Create, Alter, Drop, Truncate

Description:

- Int is used for integer values.
- VARCHAR is used for storing text.
- CREATE TABLE creates a new table named Student.

SQL

```
CREATE TABLE Student (  
  RollNo INT,  
  Name VARCHAR (50),  
  Class VARCHAR (20),  
  Marks INT  
);
```

Output:

Table Student created success-
fully

Aim: To study SQL tools and different data types used in SQL.

Theory:

SQL (Structured Query Language) is used to create and manage database.

SQL tools such as MySQL are used to execute SQL commands.

Different data types in SQL are used to store different kinds of data.

Common SQL Data types:

<u>Data type</u>	<u>Description</u>
• INT	Stores integer values
• VARCHAR	Stores text values
• CHAR	Stores fixed-length characters
• DATE	Stores date values
• FLOAT	Stores decimal numbers.

Teacher's Signature

SOL

```
CREATE TABLE Student (  
    RollNo INT  
    Name VARCHAR(50),  
    Age INT,  
    DOB DATE,  
    Percentage FLOAT  
);
```

output:

Table Student created successfully

Aim:

To create a table schema and insert records into the table using SQL commands.

Theory:

A Schema defines the structure of a table. The INSERT INTO command is used to add records into a table.

Data Description:

- CREATE TABLE creates the Employee table.
- INSERT INTO adds records into the table.

Teacher's Signature _____

SQL

```
CREATE TABLE Employee (
    Emp ID INT,
    Name VARCHAR(50),
    Department VARCHAR(30),
    Salary INT
);
```

```
INSERT INTO Employee VALUES
(1, 'Ali', 'HR', 25000),
(2, 'Ahmed', 'IT', 40000),
(3, 'Zaid', 'Finance', 35000),
(4, 'Umar', 'Marketing', 30000),
(5, 'Bilal', 'Sales', 28000);
```

Output

Emp ID	Name	Department	Salary
1	Ali	HR	25000
2	Ahmed	IT	40000
3	Zaid	Finance	35000
4	Umar	Marketing	30000
5	Bilal	Sales	28000

Aim:

To study ALTER TABLE and DROP TABLE commands in SQL.

Theory:

The ALTER TABLE command is used to modify the structure of an existing table.

The ADD col clause adds a new column, while the MODIFY clause changes the datatype or size of a column.

The DROP TABLE command deletes a table permanently.

Description:

- ADD adds a new column named Address.
- MODIFY changes the size of the Name column.
- DROP TABLE deletes the Student table.

SOL

```
CREATE TABLE Student (  
  RollNo INT,  
  Name VARCHAR (30),  
  Marks INT  
);
```

```
ALTER TABLE Student  
ADD Address VARCHAR (50);
```

```
ALTER TABLE Student  
MODIFY Name VARCHAR (50);
```

```
DROP TABLE Student;
```

Aim:
10 Study DROP, RENAME, TRUNCATE, and DESCRIBE commands in SQL.

Theory:
These SQL commands are used for managing tables in a database.

- DROP deletes a table permanently.
- RENAME changes the table name.
- TRUNCATE removes all records from a table.
- DESCRIBE displays the structure of a table.

SQL

```
CREATE TABLE Employee (  
    EmpID INT,  
    Name VARCHAR(50),  
    Salary INT  
);
```

```
DESCRIBE Employee;
```

```
RENAME TABLE Employee TO Staff;
```

```
TRUNCATE TABLE Staff;
```

```
DROP TABLE Staff;
```


Aim:

To Study different SQL constraints used in database tables.

Theory:

Constraints are ~~some~~ rules applied to table columns to maintain accuracy and integrity of data.

Common SQL Constraints:

- NOT NULL → prevents null values
- UNIQUE → allows only unique values
- Primary Key → uniquely identifies each record.
- FOREIGN KEY → links two tables
- CHECK → limits values in a column
- Default → sets a default value.

```
CREATE TABLE Department (  
  DeptID INT PRIMARY KEY,  
  DeptName VARCHAR(30) UNIQUE  
);
```

```
CREATE TABLE Employee (  
  EmpID INT PRIMARY KEY,  
  Name VARCHAR(50) NOT NULL,  
  Age INT CHECK (Age >= 18),  
  Salary INT DEFAULT 20000,  
  DeptID INT,  
  FOREIGN KEY (DeptID)  
  REFERENCES Department (DeptID)  
);
```

Aim:

To Study DML Commands in SQL.

Theory:

DML (Data Manipulation Language) commands are used to manage data in database tables.

Main commands in DML:

- INSERT → adds records
- SELECT → displays records
- UPDATE → modifies records
- DELETE → removes records.

```
CREATE TABLE Student (  
    RollNo INT,  
    Name VARCHAR(50),  
    Marks INT
```

```
);  
INSERT INTO Student VALUES
```

```
(1, 'Ali', 85),  
(2, 'Ahmed', 90),  
(3, 'Zaid', 78);
```

```
SELECT * FROM Student;
```

```
UPDATE Student  
SET Marks = 95  
WHERE RollNo = 2;
```

```
DELETE FROM Student  
WHERE RollNo = 3;
```


Aim:

To study the use of WHERE, IN, BETWEEN, and LIKE clause in SQL.

Theory:

These clauses are used with the SELECT command to filter records from a table.

- WHERE → filters records based on column's conditions.
- IN → checks multiple values
- BETWEEN → selects values within a range
- LIKE → searches patterns in text.

Description:

- WHERE filters records with marks greater than 80.
- IN selects students from Delhi & Mumbai.
- BETWEEN selects marks between 70 and 90.
- LIKE 'A%' selects names starting with letter A.

Teacher's Signature

```
CREATE TABLE Student (
```

```
RollNO INT,  
Name VARCHAR(50),  
Marks INT,  
City VARCHAR(30)
```

```
);  
Insert INTO Student VALUES
```

```
(1, 'Ali', 85, 'Delhi'),  
(2, 'Ahmed', 90, 'Mumbai'),  
(3, 'Zaid', 78, 'Delhi'),  
(4, 'Umas', 67, 'Pune'),  
(5, 'Bilal', 95, 'Mumbai');
```

```
SELECT * FROM Student  
WHERE Marks > 80;
```

```
Select * from student  
Where City IN ('Delhi', 'Mumbai');
```

```
SELECT * FROM Student  
WHERE Marks BETWEEN 70 AND 90;
```

```
SELECT * FROM Student  
WHERE Name LIKE 'A%';
```

Expt. No.

9

Aim: To write an SQL query that returns the project number and project name for projects having budget greater than 100000.

Theory: The SELECT command is used to retrieve data from a table. The WHERE clause is used to filter records according to a specified condition.

In the practical, records are selected from the PROJECT table where the budget is greater than 100000.

Teacher's Signature

SOL

```
CREATE TABLE PROJECT (  
    Pno INT,  
    Pname VARCHAR(50),  
    budget INT,  
    Ono VARCHAR(10)  
);
```

```
INSERT INTO PROJECT VALUES  
(101, 'Banking System', 150000, '01'),  
(102, 'Library System', 80000, '02'),  
(103, 'Hospital System', 200000, '03');
```

```
SELECT Pno, Pname  
FROM PROJECT  
WHERE budget > 100000;
```


Aim: TO Study the use of AND, OR, and NOT operators in SQL queries.

Theory: logical operators are used in SQL to combine multiple conditions.

- AND $\rightarrow$ Returns records ~~are~~ used when all conditions are true
- OR $\rightarrow$ Returns records when any condition is true
- NOT $\rightarrow$ Returns records when the condition is false.

CREATE Table Employee (

Reference Page 3 Expt. No. 3

Select * from Employee
Where Department = 'IT' And Salary
> 30000;

Select * from Employee
Where Department = "HR" OR
Salary > 35000;

Select * from Employee
Where NOT Department = 'Finance';

Aim: To perform aggregate functions in SQL using COUNT, SUM, AVG, MAX, MIN, GROUP BY, and Having clauses.

Theory: Aggregate functions are used to perform calculations on multiple rows of a table and return a single value.

- Count() → Counts rows
- Sum() → Adds values
- Avg() → Finds average
- Max() → Finds maximum value
- Min() → Finds minimum value
- Group By → Groups rows with same values
- Having → Applies condition on groups

Table Used
Emp (eno, ename, title, salary, dno)

1. Count function

```
SELECT COUNT(*) AS Total-Employees  
FROM Emp;
```

2. Sum Function

```
SELECT SUM(salary) AS Total-Salary  
FROM Emp;
```

3. AVG Function

```
SELECT AVG(salary) AS Average-Salary  
FROM Emp;
```

4. Max | SELECT Max(salary) AS Highest-Salary
FROM Emp;

5. Min | SELECT Min(salary) AS Lowest-Salary
FROM Emp;

6. Group By | SELECT dno, COUNT(*) AS Total-
Employees FROM Emp
GROUP BY dno;

7. Having | SELECT dno, AVG(salary) AS
Avg-Salary FROM Emp
GROUP BY dno
Having AVG(salary) > 3000;

Aim: To Study and Perform INNER JOIN and LEFT JOIN operations in SQL.

Theory: A JOIN in SQL is used to combine records from two or more tables using a common field. Joins are very important in relational database because data is usually stored in different tables.

INNER JOIN: Returns only those records whose matching values are present in both tables. Non-matching records are not displayed.

LEFT JOIN: Returns all records from the left table and the matching records from the right table. If no matching record exists in the right table, NULL values are shown.

```
CREATE Table Dept (  
  dno VARCHAR (6) PRIMARY KEY,  
  dname VARCHAR (30)
```

```
);  
CREATE TABLE EMP (  
  eno INT PRIMARY KEY,  
  ename VARCHAR (30),  
  dno VARCHAR (10)
```

```
);  
Insert Into Emp Values  
(1, 'Ali', 'D1'),  
(2, 'Sara', 'D2'),  
(3, 'John', NULL);
```

```
Select Emp.eno, Emp.ename, Dept.dname  
From Emp INNER JOIN Dept  
ON Emp.dno = Dept.dno;
```

```
SELECT Emp.eno, Emp.ename, Dept.dname  
FROM Emp  
LEFT JOIN Dept  
ON Emp.dno = Dept.dno;
```

Aim To study and ~~per~~ perform RIGHT JOIN and CROSS JOIN operations in SQL.

Theory Joins are used to combine data from multiple tables based on a common field.

RIGHT JOIN: Displays all records from the right table and matching records from the left table. If no match is found in the left table, NULL values are displayed.

CROSS JOIN: Displays the Cartesian product of two tables, it combines every row of the first table with every row of the second table.

These Joins are useful for retrieving related data from tables.

CREATE TABLE DEPT (
dno VARCHAR(10),
dname VARCHAR(20)
);

CREATE TABLE EMP(
eno INT,
ename VARCHAR(20),
job VARCHAR(10)
);

INSERT INTO DEPT VALUES
(1, 'Ali', 'D1')
(2, 'Sara', 'D2');

-- Right Join

SELECT ename, dname FROM EMP
RIGHT JOIN DEPT ON EMP.dno = DEPT.dno;

-- CROSS Join

SELECT ename, dname FROM EMP
CROSS JOIN DEPT;

Aim: To display the details of employees whose working hours are less than 10 and responsibility is Manager.

Theory: The WHERE clause in SQL is used to apply conditions on records. Multiple conditions can be used with logical operators like AND.

- AND operator returns records only when all conditions are true.
- This query helps in filtering specific records from a table.

```
CREATE TABLE WORKSON (  
  P10 INT,  
  P20 INT,  
  RESP VARCHAR(20),  
  HOURS INT  
);
```

```
INSERT INTO WORKSON Values  
(1, 101, 'Manager', 8),  
(2, 102, 'Engineer', 12),  
(3, 103, 'Manager', 6);
```

```
SELECT * FROM WORKSON  
WHERE HOURS < 10 AND  
RESP = 'Manager';
```

Aim: To display employees whose title is FE or SA and salary is greater than 35000.

Theory: The WHERE clause is used to filter records based on conditions.

- OR operator is used when any one condition can be true.
- AND operator is used when all conditions must be true.

This query helps in retrieving specific employee records based on title and salary.

```
CREATE TABLE EMP /  
  empno INT,  
  ename VARCHAR(10),  
  title VARCHAR(10),  
  salary INT  
);
```

```
INSERT INTO EMP VALUES  
(1, 'Ali', 'EE', 40000),  
(2, 'Sara', 'SA', 45000),  
(3, 'John', 'Manager', 30000);
```

```
SELECT * FROM EMP  
WHERE (title = 'EE' OR title  
= 'SA') AND salary > 35000;
```


Aim:

Dq To display employee of department ordered by decreasing salary.

Theory:

The WHERE clause is used to select specific records from a table. The order by clause is used to arrange records in ascending or descending order.

- DESC is used for descending order.
- This query helps in sorting employee records based on salary.

```
CREATE TABLE EMP /  
eno INT,  
ename VARCHAR(20),  
Salary INT,  
dno VARCHAR(20)  
/;
```

```
INSERT INTO EMP VALUES  
(1, 'Ali', 50000, 'D1')  
(2, 'Sara', 40000, 'D2')  
(3, 'John', 30000, 'D1');
```

```
SELECT * FROM EMP WHERE  
dno = 'D1' ORDER BY Salary DESC;
```

Aim:

To display department records ordered by ascending department name.

Theory:

The order by clause in SQL is used to sort records in ascending or descending order.

- ASC is used for ascending order.
- By default, records are arranged in ascending order.

This query helps in organizing department data alphabetically.

```
CREATE TABLE DEPT (  
  dno VARCHAR (10),  
  dname VARCHAR (20).  
);
```

```
INSERT INTO DEPT VALUES  
( 'D1', 'HR' ),  
( 'D2', 'Sales' ),  
( 'D3', 'Consulting' );
```

```
SELECT * FROM DEPT ORDER BY  
dname ASC;
```


Aim:

To display project name, hours worked, and project number where working are greater than 10.

Theory:

The WHERE clause is used to filter records based on a condition.

- It helps in displaying only required records from a table.

- Conditions can be applied using comparison operators such as $>$, $<$, $=$ etc

This query retrieves project details where working hours are greater than 10.

```
CREATE TABLE WORKSON (  
  Pro INT,  
  Pname VARCHAR(20),  
  hours INT  
);
```

```
INSERT INTO WORKSON VALUES  
(101, 'Website', 12),  
(102, 'APP', 8),  
(103, 'Database', 15);
```

```
SELECT Pname, hours, Pro  
FROM WORKSON  
WHERE hours > 10;
```